RAG-Based Customer Support Assistant

---

1. Introduction

1.1 Purpose

This document defines the High-Level Design for a Retrieval-Augmented Generation (RAG) based Customer Support Assistant. The system is designed to deliver accurate, context-aware responses by grounding LLM outputs in a document-based knowledge base and controlling response behavior through workflow-driven decision logic.

1.2 Scope

In Scope

- PDF-based knowledge ingestion
- Semantic retrieval using embeddings
- Context-grounded response generation
- Workflow-based decision routing
- Human-in-the-Loop (HITL) escalation

Out of Scope

- Authentication and authorization
- Multi-tenant deployment
- Distributed infrastructure
- Continuous learning or retraining

---

2. System Overview

2.1 Problem Statement

Customer support systems frequently encounter repetitive queries whose answers exist in documentation. Traditional AI systems often generate ungrounded or inconsistent responses, leading to reduced reliability.

2.2 Solution Overview

The system implements a controlled RAG architecture where:

- user queries are matched against a vectorized knowledge base
- relevant context is retrieved using semantic similarity
- responses are generated strictly from retrieved content
- a workflow engine determines whether to respond or escalate

---

2.3 Objectives

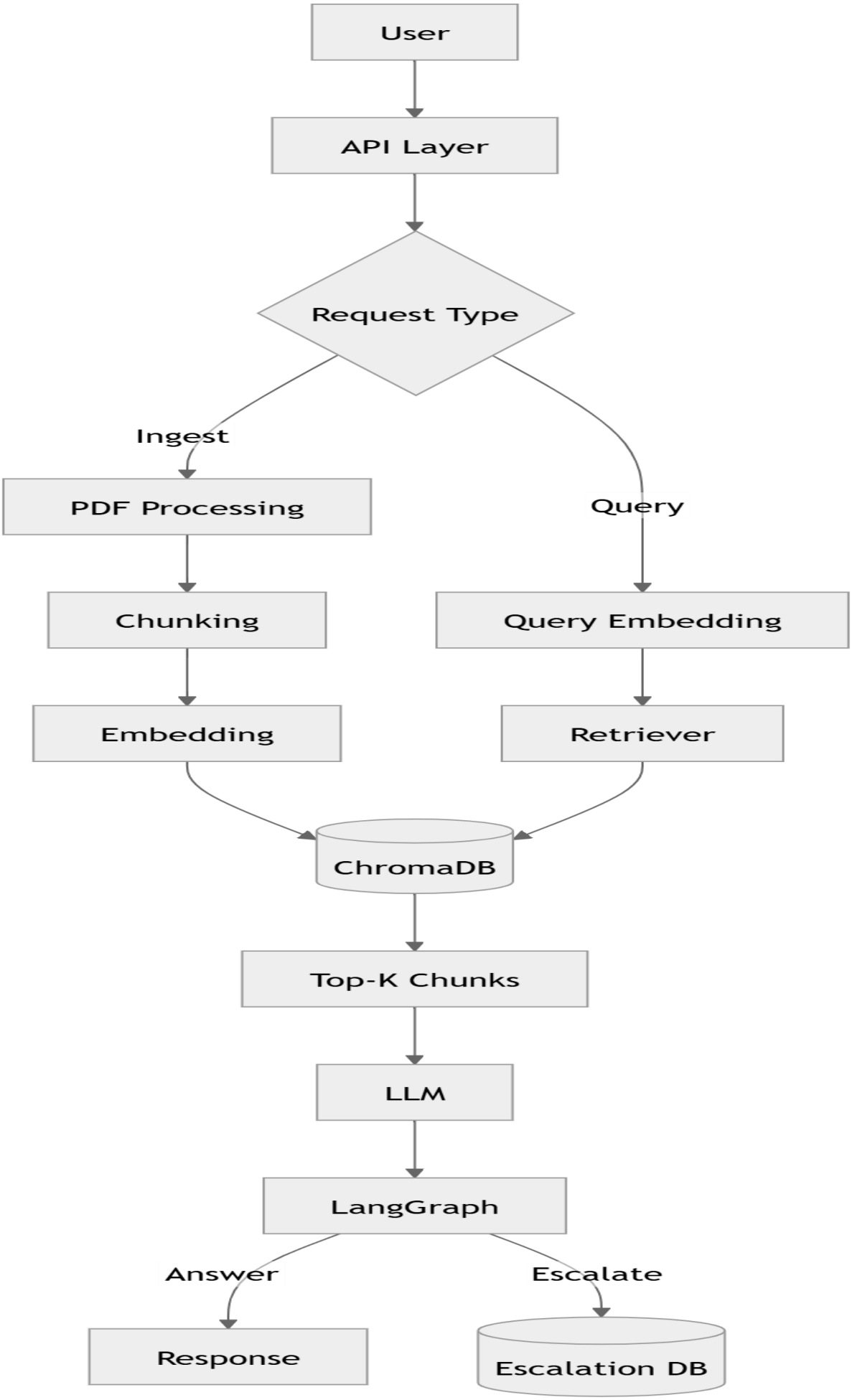
- ensure response accuracy through retrieval grounding
- reduce hallucination in LLM outputs
- automate resolution of repetitive queries
- enable controlled escalation for uncertain cases
- maintain traceability via citations and structured outputs

---

3. Architecture Overview

The system follows a **pipeline-driven architecture with workflow orchestration**, consisting of two primary flows:

- ingestion pipeline
- query and decision pipeline


User
API Layer
Request Type
Ingest
Query
PDF Processing
Chunking
Embedding
Query Embedding
Retriever
ChromaDB
Top-K Chunks
LLM
LangGraph
Answer
Escalate
Response
Escalation DB

3.1 Core Components

1. API Layer

Handles request routing, validation, and response formatting.

---

2. Document Ingestion Layer

Processes PDF documents and converts them into structured, retrievable data.

---

3. Embedding Layer

Generates vector representations for both document chunks and user queries.

---

4. Vector Storage Layer

Stores embeddings and metadata in a persistent vector database.

---

5. Retrieval Layer

Fetches top-K relevant chunks based on semantic similarity.

---

6. LLM Processing Layer

Generates grounded responses using retrieved context.

---

7. Workflow Engine

Controls execution flow and decision routing using a graph-based model.

---

8. Escalation Layer (HITL)

Handles uncertain cases by creating and managing escalation tickets.

---

4. Architecture Flow

User Request

↓

API Layer

↓

Query Processing

↓

Embedding Generation

↓

Retriever (Vector DB)

↓

Top-K Context

↓

LLM Processing

↓

Workflow Decision

→ Return Answer

→ Create Escalation

---

5. Data Flow

5.1 Ingestion Flow

1. PDF document is uploaded
2. text is extracted page-wise
3. text is normalized and segmented into chunks
4. embeddings are generated for each chunk
5. embeddings and metadata are stored in vector database

5.2 Query Flow

1. user submits a query
2. query is embedded
3. retriever fetches top-K relevant chunks
4. retrieved context is passed to LLM
5. LLM generates structured response
6. workflow evaluates decision criteria
7. system returns answer or triggers escalation

---

6. Component Responsibilities

6.1 API Layer

- accepts ingestion and query requests
- validates input
- returns structured responses

6.2 Ingestion Layer

- extracts text from PDFs
- prepares chunked data
- initiates embedding generation

6.3 Embedding Layer

- converts text into vector format
- ensures consistent representation for retrieval

6.4 Vector Storage

- persists embeddings

- supports similarity-based retrieval

6.5 Retrieval Layer

- performs semantic search
- returns ranked chunks with relevance scores

6.6 LLM Layer

- consumes query + retrieved context
- generates grounded response
- outputs confidence and escalation signal

6.7 Workflow Engine

- evaluates system state
- applies routing logic
- determines final outcome

6.8 Escalation Layer

- stores escalation tickets
- supports human review and resolution

---

7. Decision Logic

7.1 Answer Conditions

The system returns an automated response when:

- relevant context is retrieved
- retrieval confidence meets threshold
- LLM confidence is above threshold

- no escalation signal is present

7.2 Escalation Conditions

The system triggers escalation when:

- no relevant context is available
- retrieval confidence is below threshold
- LLM confidence is insufficient
- query cannot be resolved from knowledge base

---

8. Knowledge Base Design

8.1 Structure

Each knowledge unit contains:

- query
- contextual information
- response
- conditions
- escalation rules

8.2 Chunking Strategy

- fixed-size segments
- overlapping context windows
- maintained source traceability

8.3 Metadata

Each stored chunk includes:

- document reference

- page number
- chunk index
- embedding vector

---

9. Human-in-the-Loop (HITL)

9.1 Purpose

To ensure safe handling of low-confidence or unsupported queries.

9.2 Escalation Flow

1. system detects escalation condition
2. escalation ticket is created
3. ticket stored with context and draft response
4. human agent reviews and resolves
5. resolution is recorded

---

10. External Interfaces

API Endpoints

- POST /knowledge-bases/ingest → ingest PDF
- POST /support/query → process query
- GET /escalations → list escalation tickets
- POST /escalations/{id}/resolve → resolve escalation

---

11. Non-Functional Considerations

Performance

- low-latency retrieval and response generation

Scalability

- supports increased query load through modular architecture

Reliability

- controlled fallback through escalation
- avoids hallucinated responses

---

12. Assumptions & Constraints

- single knowledge base in current implementation
- no authentication layer
- no cross-session memory
- reliance on external LLM API

---

13. Risks & Mitigation

Risk	Mitigation
hallucinated responses	strict retrieval grounding
low-quality retrieval	top-K search with thresholds
incorrect routing	confidence-based decision logic
unsupported queries	escalation mechanism

---

14. Conclusion

This system provides a structured and controlled approach to AI-powered customer support by integrating:

- retrieval-based grounding
- LLM-based response generation

- workflow-driven decision control
- human escalation mechanisms

The architecture ensures reliability, traceability, and practical applicability in real-world support environments.